

ASSIGNMENT 5 — HTML Web Table Extractor

1. Experiment Title

HTML Web Table Data Extraction Using Selenium

2. Problem Statement

Automate the extraction of information from a multi-column HTML web table. The script should iterate through the table rows and columns, identify a specific row by matching a name, and retrieve the corresponding value from the required column.

3. Objective

- To locate an HTML web table using Selenium.
- To iterate through table rows and columns.
- To identify a particular row using a name.
- To retrieve the corresponding Status or Price value.
- To demonstrate structured web-data extraction.

4. Tools, Software and Concepts Used

Tools & Software:

- Python
- Selenium WebDriver
- Google Chrome
- VS Code / PyCharm

Concepts:

- HTML Tables
- Rows and Columns
- XPath
- find_elements()
- Iteration

- Text Extraction
- Selenium WebDriver

5. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

def test_web_table_extractor():
    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.get("https://the-internet.herokuapp.com/tables")
    rows = driver.find_elements(
        By.XPATH, "//table[1]/tbody/tr"
    )
    target_name = "John"
    target_due = None
    for row in rows:
        columns = row.find_elements(By.TAG_NAME, "td")

        # Iterate through columns
        for column in columns:

            # Find the target name
            if column.text == target_name:

                # Due is the 4th column
```

```
target_due = columns[3].text
```

```
print("Name:", target_name)
```

```
print("Due:", target_due)
```

```
break
```

```
if target_due is not None:
```

```
break
```

```
assert target_due is not None
```

```
assert target_due == "$50.00"
```

```
time.sleep(10)
```

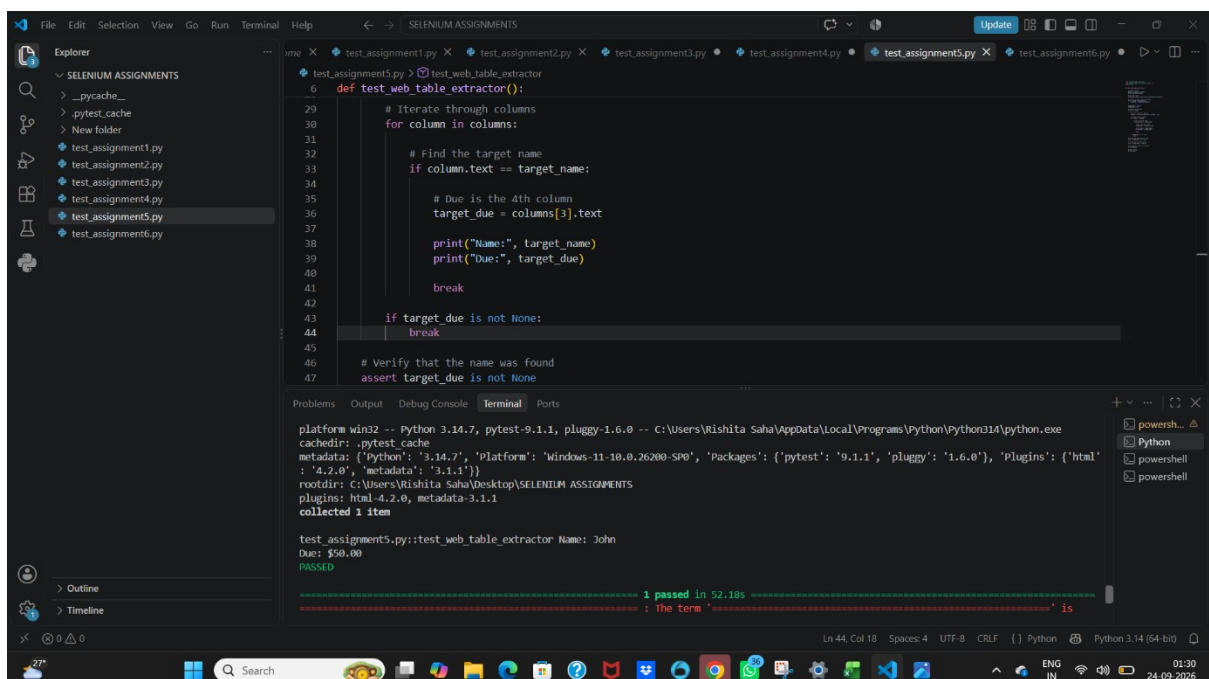
```
# Close browser
```

```
driver.quit()
```

Execution

```
python test_assignment5.py
```

6. Output with Clearly Labelled Screenshots



The screenshot displays a Visual Studio Code window with a file explorer on the left showing a directory named 'SELENIUM ASSIGNMENTS'. The main editor area contains a Python script named 'test_web_table_extractor()' with the following code:

```
def test_web_table_extractor():
    # Iterate through columns
    for column in columns:
        # Find the target name
        if column.text == target_name:
            # Due is the 4th column
            target_due = columns[3].text
            print("Name:", target_name)
            print("Due:", target_due)
            break
    if target_due is not None:
        break
    # Verify that the name was found
    assert target_due is not None
```

The bottom panel shows the 'Terminal' output, which includes the command used to run the script and the resulting output:

```
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Rishita Saha\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
metadata: {'python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0', 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS
plugins: html-4.2.0, metadata-3.1.1
collected 1 item

test_assignment5.py::test_web_table_extractor Name: John
Due: $50.00
PASSED

===== 1 passed in 52.18s =====
: the term '-----' is
```

The status bar at the bottom indicates the file is at line 44, column 18, with 4 spaces, in UTF-8 encoding, with CRLF line endings, using the Python interpreter at C:\Users\Rishita Saha\AppData\Local\Programs\Python\Python314\python.exe.

Extracted Value

Show the terminal displaying the corresponding value.

7. Result

The HTML table was successfully accessed using Selenium. The script iterated through the rows, identified the required name, and extracted the corresponding value from the specified column.

8. Observation

- Table rows can be accessed using XPath.
- Individual cells can be obtained using `find_elements()`.
- Iterating through rows makes it possible to locate a specific record dynamically.
- The required value can be extracted from the matching row.

9. Conclusion

The experiment successfully demonstrated **HTML web table extraction using Selenium WebDriver**, including locating a specific record and retrieving its corresponding table value.